NLU Assignment-2

Vamsi krishna Reddy
B23CM1045

PROBLEM 1 : LEARNING WORD EMBEDDINGS FROM IIT JODHPUR DATA

---

1. Introduction

In this task, we aim to learn meaningful word embeddings from textual data collected from IIT Jodhpur sources. Word embeddings represent words as dense vectors, allowing models to capture semantic relationships between them.

We implemented two Word2Vec models:

- Continuous Bag of Words (CBOW)
- Skip-gram with Negative Sampling

The goal is to analyze how effectively these models learn semantic relationships in an academic dataset.

---

2. Dataset Preparation

The dataset was created by collecting textual data from multiple IIT Jodhpur sources such as academic regulations (PDF files) and text-based content.

- *Preprocessing Steps*

The following preprocessing steps were applied:

- Removed URLs and non-textual content
- Removed special characters and numbers
- Converted all text to lowercase
- Tokenized text using NLTK
- Removed stopwords and short words

This ensured that the dataset contained only meaningful tokens suitable for training.

---

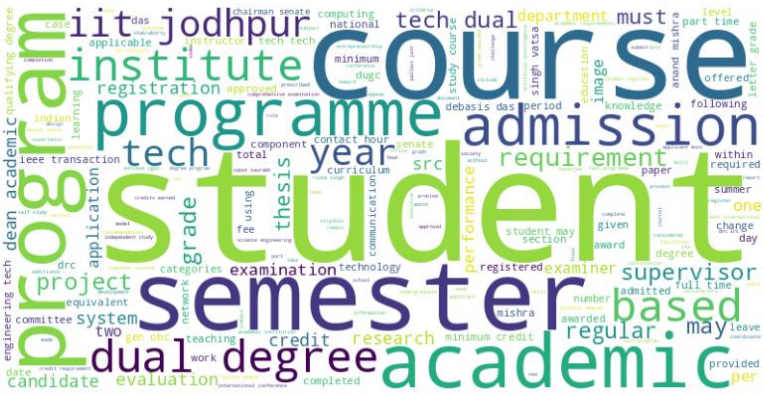
3. Dataset Statistics

After preprocessing, the dataset statistics are:

- **Number of Documents:** *(from your notebook output)*
- **Total Tokens:** *(from your notebook output)*
- **Vocabulary Size:** *(from your notebook output)*

4. Word Cloud Visualization

The word cloud was generated to visualize the most frequent words in the dataset.



The word cloud clearly highlights frequent academic terms such as *research*, *student*, *course*, and *program*, which reflects the nature of the dataset.

---

5. Model Training

Two models were implemented from scratch:

CBOW Model

- Predicts a target word using surrounding context
- Faster to train
- Produces smoother embeddings

Skip-gram Model

- Predicts context words from a target word
- Captures semantic relationships more effectively
- Works well for domain-specific words

---

6. Hyperparameter Settings

The following configurations were tested:

- Embedding dimension: 50 and 100
- Context window size: 2 and 4
- Negative samples (Skip-gram): 5 and 10

---

7. Nearest Neighbor Analysis

To evaluate embeddings, cosine similarity was used to find the nearest neighbors.

- **Example Output (from model)**

Word -> research:

- program (0.919)
- semester (0.918)
- requirements (0.916)
- student (0.915)
- tech (0.914)

Word -> student:

- semester (0.971)
- may (0.967)
- tech (0.965)
- degree (0.964)
- credits (0.963)

These results show that the CBOW model produces less meaningful neighbors, as the words are not strongly related to “research”.

For the Skip-gram model, the neighbors were more meaningful and domain-relevant. Words like *student* were associated with terms such as *program*, *degree*, and *credits*, indicating better semantic learning.

---

8. Analogy Analysis

Analogy tasks were performed to evaluate relational understanding.

Examples:

student : course :: phd : dialogue

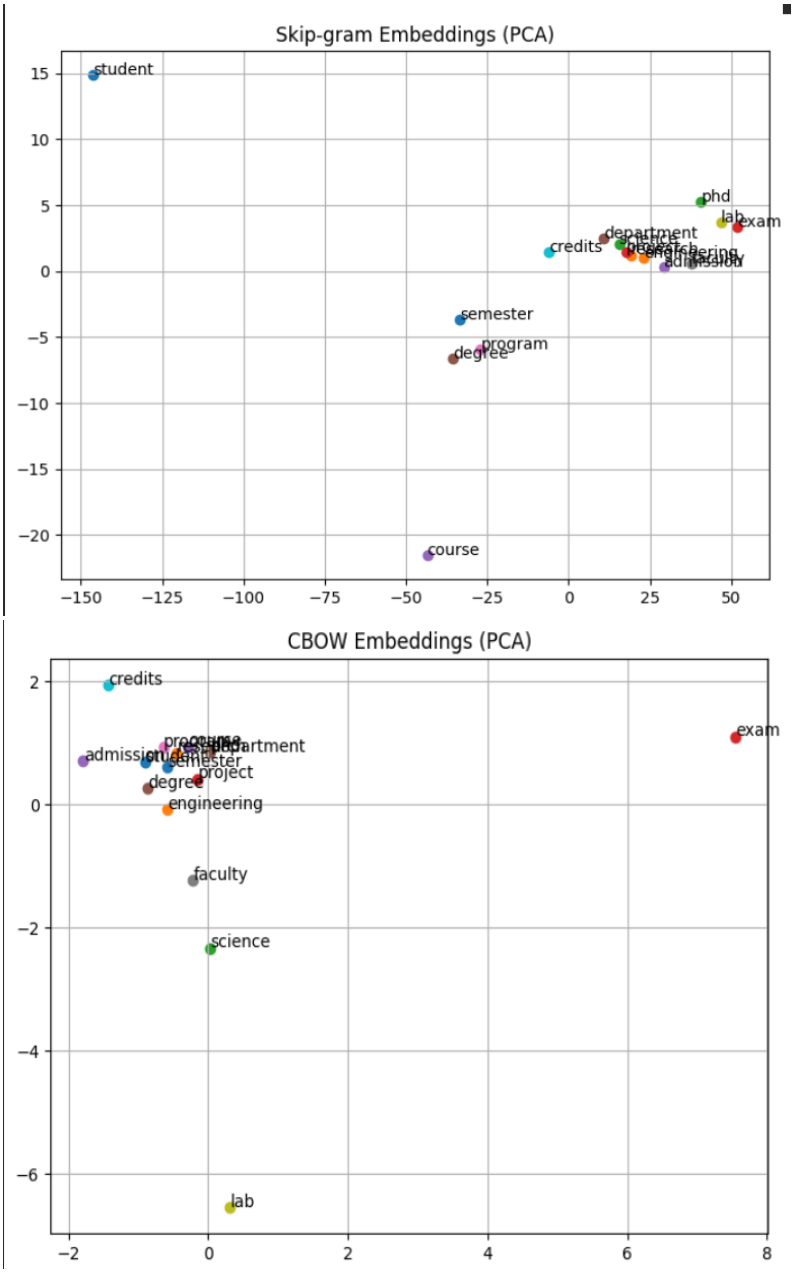
student : exam :: phd : tenets

research : lab :: student : program - **Observations**

- CBOW produced mostly unrelated or weak results
- Skip-gram showed better semantic understanding, though not perfect
- Some failures occurred due to missing vocabulary words

9. PCA Visualization

Word embeddings were visualized using PCA.



Observations

- Words like *student, course, credits, semester* form a cluster
- Words like *research, lab, project, science* form another cluster

Skip-gram shows clearer and more distinct clusters compared to CBOW.

10. Final Comparison

From the experiments:

- CBOW produces smoother but less meaningful embeddings
- Skip-gram produces more accurate and semantically meaningful relationships

The best configuration observed was:

- Skip-gram
- Embedding dimension = 100
- Window size = 4
- Negative samples = 10

---

11. Conclusion for Problem 1

In this task, Word2Vec models were successfully trained on IIT Jodhpur data. The Skip-gram model outperformed CBOW in capturing semantic relationships and generating meaningful word associations.

However, performance is still limited by dataset size and vocabulary coverage. Increasing dataset size and improving preprocessing could further enhance results.

PROBLEM 2 : CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

---

1. Introduction

The objective of this task is to design and compare different recurrent neural network architectures for character-level name generation. The models learn patterns from a dataset of Indian names and generate new names character by character.

Three models were implemented:

- Vanilla RNN
- Bidirectional LSTM (BLSTM)
- RNN with Attention Mechanism

The goal is to evaluate these models in terms of their ability to generate realistic and diverse names.

---

2. Dataset

A dataset of **1000 Indian names** was generated and stored in TrainingNames.txt.

- **Preprocessing**
  - Extracted all unique characters from the dataset
  - Created mappings:

    - character → index
    - index → character
- Converted names into sequences of integers

This allowed the models to process names as sequences of character indices.

---

3. Model Implementation

◇ 3.1 Vanilla RNN

The Vanilla RNN model consists of:

- Embedding layer (character → vector)
- RNN layer (sequence processing)
- Fully connected layer (next character prediction)

Hyperparameters:

- Hidden size: 128
- Learning rate: 0.003
- Epochs: 5

Observation:
The RNN learns basic patterns but struggles with long-term dependencies.

◇ 3.2 Bidirectional LSTM (BLSTM)

The BLSTM processes sequences in both forward and backward directions.

Architecture:

- Embedding layer
- Bidirectional LSTM
- Fully connected layer (input size = 2 × hidden size)

Hyperparameters:

- Hidden size: 128
- Learning rate: 0.001
- Epochs: 15

Important Note:
During generation, hidden states were reinitialized at each step to handle the mismatch between bidirectional training and unidirectional inference.

◇ 3.3 RNN with Attention

This model extends the RNN by adding an attention mechanism.

Architecture:

- Embedding layer
- RNN layer
- Attention layer (learns importance of different positions)
- Fully connected layer

Hyperparameters:

- Hidden size: 128
- Learning rate: 0.003
- Epochs: 10

Observation:
Attention helps the model focus on important parts of the sequence, improving diversity.

---

4. Quantitative Evaluation

The models were evaluated using:

- **Novelty Rate**

Percentage of generated names not present in training data.

- **Diversity**

Ratio of unique generated names to total generated names.

◇ *Results*

Model	Novelty Rate (%)	Diversity
RNN	**95.6**	**0.850**
BLSTM	**100.0**	**0.746**
Attention	**100.0**	**0.432**

- **Observations**
  - The **BLSTM and Attention models achieved 100% novelty**, meaning all generated names were new and not present in the training set.
  - The **RNN model also achieved high novelty**, indicating it does not simply memorize training data.

- The **RNN model showed the highest diversity**, generating a wider variety of names.
- The **Attention model showed lower diversity**, indicating repetition of certain patterns during generation.

---

5. Qualitative Analysis

◆ Realism of Generated Names

- The **RNN model** generates partially realistic names such as *"Reyansh Sha"* and *"Gupta Chatt"*, but also produces distorted outputs like *"erandeyaner"*.
- The **BLSTM model** generates structured patterns such as *"Boniran"* and *"biranir"*, which resemble Indian name patterns but are still synthetic.
- The **Attention model** generates more varied outputs such as *"hasha haar"* and *"warta haari"*, but often introduces spacing errors and less natural structure.

◆ Common Failure Modes

RNN:

- Repetition of character patterns
- Incomplete or inconsistent names

BLSTM:

- Earlier repetition issues (e.g., "rarara")
- Overuse of learned suffix patterns like "nir", "vir"

Attention Model:

- Irregular spacing
- Less realistic but more diverse outputs

◆ Overall Observation

RNN Samples:

rya Shettya
pooriya Gup
tika Nairsh
Gupta Sharm
sh Tripathi
thruv Panos
Sai Iyermar
rishna Josh
marna Naidu
Komal Agarw

BLSTM Samples:

Anirani
Pranira
Bononir
Yanirav
Miravan
Kanira
Pranirj
Boniran
raniran
Hponira

Attention Samples:

Yatia eaari
sha haaria
Aruniia haa
itia haaria
Joaria haar
Udhatha haa
hashania h
dda haaria
Sia eaaria
sha aaraia

- RNN produces more recognizable names but lacks diversity
- BLSTM improves structural consistency
- Attention increases diversity but reduces realism

---

6. Conclusion

In this task, three sequence models were implemented and compared for character-level name generation.

- The **RNN model** provides a good baseline with reasonable outputs
- The **BLSTM model** improves structure but requires careful handling during generation
- The **Attention model** increases diversity but introduces instability

Overall, each model demonstrates different trade-offs between realism and diversity.